Overview – Region proposal

Context:

This stage doesn't know **what** objects are in the image, it only guesses **where** something might be. A brute-force search over all possible rectangles (x, y, width, height) is infeasible, which is why we use **region proposal methods**.

There are multiple method for region proposal, such as:

- **Selective search**: used in R-CNN and Fast R-CNN.
- **Region proposal network (RPN)**: used in Faster R-CNN.

How does the selective search work?

The selective search is composed of five steps.

1) Oversegmentation

The CNN uses an algorithm like **Felzenszwalb and Huttenlocher's graph-based segmentation** to break the image into **superpixels** — small regions that are **locally homogeneous** in color and texture, that respect **image boundaries** (don't cross strong edges), and that are small enough to allow **later merging at multiple scales**.

These superpixels are our "**atomic parts**" — the smallest units we can merge.

What is Felzenszwalb & Huttenlocher approach ?

Felzenszwalb & Huttenlocher approach segmentation as a **graph problem**.

A) Graph construction

- The **nodes** are representing each pixel of the image.
- The **edges** connect each pixel to its 4 or 8 neighbors (↑↓←→ + 4 optionally diagonals).
- The **edge weight** represents the **dissimilarity** between pixels u and v, and it is calculated as follows:

$$\mathrm{w}(\mathrm{u},\mathrm{v}) = \|\mathrm{I}(\mathrm{u}) - \mathrm{I}(\mathrm{v})\|$$

  Where:

  - $\mathrm{I}(\mathrm{u})$ and $\mathrm{I}(\mathrm{v})$ are the RGB color vectors for each pixel u and v.
  - $\|\cdot\|$ is the Euclidean distance.

- The algorithm then **sorts** all edges $w(u, v)$ in **non-decreasing order**, and merges nodes into regions also called "**components**" if their similarity is high — but stops merging when regions are too different internally.

B) Initialization

- Initially, each pixel is its own component.

- Each component has an **internal difference**:

$$\mathrm{Int}(\mathrm{C}) = \max_{(\mathrm{u},\mathrm{v})\in \mathrm{MST}(\mathrm{C})} \mathrm{w}(\mathrm{u},\mathrm{v})$$

  Where :

  - MST(C) = the **minimum spanning tree** (which is a subset of edges that connects all the pixels in the component – here the only node C, that contains no **cycles**, and that has exactly C-1 edges for C pixels) of the edges within component C.
- Initially, Int(C) = 0 because C is a component that contains only one pixel.

C) Merge criterion

- Then, the method applies a **merge criterion**.
- For each edge $w(u, v)$ in sorted order, we first take two pixels : component C1 contains the pixel u, and the component C2 contains the pixel v.
- Then, we define a **threshold function** for each component C1&C2:

$$\tau(C) = \frac{k}{|C|}$$

  Where :

  - k = user-defined parameter **controlling scale of segmentation**.
  - $|\mathrm{C}|$ = size (number of pixels) in the component C.
- The idea is that larger components cover more area, so they can tolerate less similarity when merging.
- Then we merge C1 and C2 if:

$$\mathrm{w}(\mathrm{u},\mathrm{v}) \leq \min\left(\mathrm{Int}(\mathrm{C1}) + \tau(\mathrm{C1}),\ \mathrm{Int}(\mathrm{C2}) + \tau(\mathrm{C2})\right)$$

- Thus, that method prevents merging across **strong boundaries with high edge weight**.

D) Post-processing

- Finally, there is a **post-processing** to merge very small components into neighboring regions to avoid **tiny fragments**.
- The output is a set of connected components, each considered to be a superpixel.

This produces a set of **small connected regions**, roughly aligned with **object boundaries**.

2) Descriptors

Then, the method computes **descriptors** for each superpixel, like :

- Compute its **color histogram** for the distribution of pixel colors ($\mathrm{H_s}$)
- Compute its **texture histogram** ($\mathrm{T_s}$) for edges, gradients, or filter responses
- Record its **size** ($|s|$) (number of pixels)
- Record its **bounding box** ($area\big(BB(s)\big)$)

So each region is described by a **feature vector** summarizing its appearance and geometry.

3) Region similarity

Then, the method finds **neighboring components** to construct a graph where each region is a node and where there's an edge between two regions if they are **adjacent**.

Now for each edge (a,b), we compute a **similarity score** $s(a,b)$.

As discussed before, it's the sum of **four cues**:

$$s(a,b) = s_{\text{color}} + s_{\text{texture}} + s_{\text{size}} + s_{\text{fill}}$$

Where :

- s_{color} = similarity between color histograms of two components a and b:

$$\mathrm{s_{color}(a,b)} = \sum_{\mathrm{i}} \min\left(\mathrm{H_a(i)}, \mathrm{H_b(i)}\right)$$

  Where:

  - $H_a(i)$ and $H_b(i)$ are **normalized histogram bins**.

  Two regions with **overlapping color distributions** have a high color similarity.

- s_{texture}= similarity between textures of two components a and b:

$$\mathrm{s_{texture}(a,b)} = \sum_{\mathrm{i}} \min\left(\mathrm{T_a(i)}, \mathrm{T_b(i)}\right)$$

  Where:

  - $T_a(i)$ and $T_b(i)$ are texture descriptors.

  Two regions with **similar textures** (like grass patches) get a high score.

- s_{size}= similarity between sizes of two components a and b:

$$\mathrm{s_{size}(a,b)} = 1 - \frac{|a| + |b|}{|I|}$$

  Where:

  - $|a|$ and $|b|$ are the number of pixels in each region.
  - $|I|$ is the total number of pixels in the image.

  Small regions are merged first to **simplify the segmentation hierarchy**.

- s_{fill}= similarity between fills of two components a and b:

$$\mathrm{s_{fill}(a,b)} = 1 - \frac{area\big(BB(a,b)\big) - |a| - |b|}{|I|}$$

  Where:

  - BB(a,b) is the area of the bounding box englobing both a and b components.

  Two regions that would form a **compact shape** together are merged first.

As a result, each term encourages merging regions that have similar color, similar texture, that are small, and that form a compact shape when combined.

Thus, two neighboring patches look very similar probably **belong to the same object**.

4) Hierarchical merging

Finally, the method merges the pair (a,b) with the **highest similarity score**, it removes similarities involving a or b since they're gone, and it **computes new similarities** between ab and its new neighbors.

The method iterates the entire procedure again and again until the entire image becomes one single component.

5) Bounding boxes

To get the **bounding boxes**, the algorithm looks at the graph tree. The **root node** at the top represents the entire image, while its **child nodes** represent the bounding boxes.

The output of the region proposal method is the children of the root node of the graph tree.